

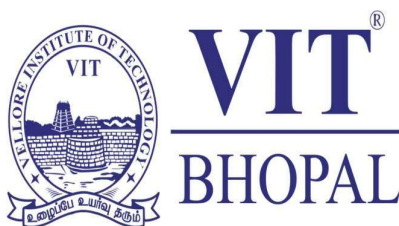
AI HEALTHCARE CENTRE

A PROJECT REPORT

Submitted by

No.	Name	Registra on No.
1	Swarnima Sah	25BAI10597
2	Naincy Chandra	25BAI10941
3	Tanishka Joshi	25BAI10767
4	Priyanshu Sajwan	25BAI11245
5	Sumit	25BAI11024
6	Varad Jadhav	25BAI10629

In partial fulfillment for the award of the degree
BACHELOR OF TECHNOLOGY
(AI & ML)



SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

VIT BHOPAL UNIVERSITY
KOTHRICALAN, SEHORE
MADHYA PRADESH – 466114

OCTOBER 2026

(A typical specimen of Bonafide Certificate)

VIT BHOPAL UNIVERSITY, KOTHRIKALAN, SEHORE
MADHYA PRADESH – 466114

BONAFIDE CERTIFICATE

Certified that this project report titled

“ AI HEALTHCARE CENTRE ” is the bonafide work of “

Swarnima Sah (25BAI10597)

Naincy Chandra (25BAI10941)

Tanishka Joshi (25BAI10767)

Priyanshu Sajwan (25BAI11245)

Sumit (25BAI11024)

Varad Jadhav (25BAI10629) ”

who carried out the project work under my supervision. Certified further that to the best of my knowledge, the work reported here does not form part of any other project/research work based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

PROJECT SUPERVISOR

Dr. Jitendra Pratap Singh Mathur

School of Computer Science and Engineering
VIT BHOPAL UNIVERSITY

ACKNOWLEDGEMENT

First and foremost I would like to thank the Lord Almighty for His presence and immense blessings throughout the project work.

I wish to express my heartfelt gratitude to Dr Jitendra Pratap Singh Mathur, Programme Chair for Artificial Intelligence and Machine Learning for much of his valuable support and encouragement in carrying out this work.

I would like to thank all the technical and teaching staff, who extended directly or indirectly all support.

Last, but not the least, I am deeply indebted to my parents who have been the greatest support while

I worked day and night for the project to make it a success.

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
ML	Machine Learning
CSV	Comma-Separated Values
UI	User Interface
API	Application Programming Interface
HTTP	Hypertext Transfer Protocol
TB	Tuberculosis
CORS	Cross-Origin Resource Sharing
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
RF	Random Forest

LIST OF FIGURES

Figure No.	Title
4.1	Overall system architecture
4.2	Respiratory screening workflow
4.3	Frontend–backend communication flow
4.4	Project folder structure
5.1	Prototype execution and prediction flow

ABSTRACT

The AI Healthcare Centre is a web-based healthcare management prototype designed to demonstrate how Artificial Intelligence and Machine Learning can be integrated into a simple healthcare application. The project focuses on preliminary symptom-based screening, with particular emphasis on common and respiratory-related symptoms. The system provides separate interfaces for hospital administration, doctors, and patients.

The machine-learning component uses a small CSV dataset containing disease labels and symptom-related attributes such as fever, cough, fatigue, difficulty breathing, age, and gender. A Random Forest classifier is trained using a preprocessing pipeline that converts categorical inputs into machine-readable features. The trained model is saved as a Python pickle file and served through a Flask backend. The HTML/JavaScript frontend communicates with the backend using an HTTP POST request and displays the returned prediction.

The project is an academic prototype and not a clinical diagnostic system. The screening output is preliminary and should not be used as a substitute for evaluation by a qualified healthcare professional. The current dataset is small and is suitable for demonstrating the software workflow rather than establishing clinical accuracy or medical validity.

TABLE OF CONTENTS

Chapter No.	Title
1	INTRODUCTION
2	LITERATURE SURVEY
3	SYSTEM ANALYSIS
4	SYSTEM DESIGN AND IMPLEMENTATION
5	PERFORMANCE ANALYSIS
6	Frontend Implementation
7	Flask Backend
8	FUTURE ENHANCEMENT AND CONCLUSION
Appendix A	Dataset Structure
Appendix B	Key Source Code Structure
References	

CHAPTER 1

INTRODUCTION

1.1 Introduction

Healthcare applications increasingly use software systems to organize information, assist users, and support preliminary analysis. Artificial Intelligence and Machine Learning provide methods for identifying patterns in data and producing predictions from user-provided inputs. This project applies these concepts to a simple web-based healthcare prototype.

Artificial Intelligence can also be used to assist with preliminary analysis of symptoms. The purpose of the AI component in this project is not to replace a doctor but to demonstrate how machine learning can process structured symptom information and provide a preliminary screening result.

The AI Healthcare Centre combines healthcare-management features and an AI-based respiratory disease screening component in one web application.

1.2 Motivation for the Work

The main motivation behind this project is to develop a centralized healthcare platform where multiple healthcare-related activities can be demonstrated through one system.

The project combines:

- Patient management
- Doctor management
- Appointment management
- Medical records
- Prescription management
- Medicine services
- AI-based respiratory screening
- Database management

This makes the project a combination of a Healthcare Management System + Machine Learning Application.

1.3 Overview of the Project

The AI Healthcare Centre contains three main user roles: Hospital Administration, Doctor, and Patient. The patient interface includes respiratory symptom screening, appointment booking, medical-record display, medicine-delivery simulation, shopping cart, and order display. The doctor interface includes patient-record access and prescription entry. The administration interface displays basic hospital statistics and management actions.

The prototype dataset contains seven columns: Disease, Fever, Cough, Fatigue, Difficulty Breathing, Age, and Gender. Disease is the target variable. A Random Forest classifier is trained using the six input features.

1.4 Problem Statement

Small-scale healthcare-management systems may require separate processes for:

- Patient registration
- Doctor management
- Appointment scheduling
- Medical records
- Prescriptions
- Symptom assessment

The proposed system provides a centralized web application where these activities can be accessed through different role-based interfaces. The system also demonstrates an AI-assisted workflow where a patient can enter basic symptoms and demographic information and receive a preliminary screening result.

1.4 Objectives

The major objectives of the project are:

Healthcare Management

To develop a centralized system for managing patients, doctors, appointments and medical information.

AI-Based Screening
To implement a machine-learning model capable of providing a preliminary disease-screening result from selected symptoms and demographic information.

Role-Based Interface

To provide separate interfaces for:

- Patients □ Doctors
- Administrators

Appointment Management

To allow appointments to be created, viewed and managed.

Medical Record Management

To store and retrieve medical-record information.

Prescription Management

To allow doctors to create prescription information containing medicine, dosage and instructions.

Medicine Service

To provide medicine browsing, shopping-cart and ordering functionality.

Database Management

To store healthcare information using SQLite.

1.5 Scope of the Project

The project consists of four major components.

Patient Module The patient can:

- Login
- Enter respiratory symptoms
- Perform AI screening
- Book appointments
- View medical records
- Browse medicines
- Add medicines to cart
- Place orders
- View previous orders

Doctor Module The doctor can:

- View patients
- View appointments
- View patient records
- Review screening information
- Create prescriptions

Administrator Module The administrator can:

- Manage doctors
- Manage patients
- Manage appointments
- View hospital statistics
- View reports

AI/ML Module

The AI component performs:

- Dataset loading
- Feature and target separation
- Data preprocessing
- Train/test splitting
- Random Forest training
- Model evaluation
- Model saving

- Prediction through Flask API

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

This chapter reviews the technical ideas used in the project: AI-assisted healthcare applications, symptom-based screening, supervised classification, data preprocessing, and web-based model deployment.

2.2 AI and ML in Healthcare

Machine learning can learn relationships between input variables and labelled outcomes. In healthcare-related research it can support classification, risk estimation, image analysis, and decision-support. In this project it is used only to demonstrate classification from a small set of symptom attributes.

2.3 Symptom-Based Screening

Symptom-based screening systems collect structured information and apply rules or learned models to generate a preliminary result. Their educational value includes demonstrating data collection, feature representation, classification, and result presentation. Screening should not be represented as a confirmed diagnosis.

2.4 Machine Learning Algorithms

The project uses Random Forest classification, an ensemble method that combines multiple decision trees. Categorical features are converted using One-Hot Encoding, while Age remains numerical.

Technique	Role
One-Hot Encoding	Converts categorical values into numerical features.
Random Forest	Classifies the input into a disease label.
Pipeline	Combines preprocessing and prediction.
Flask	Provides the prediction API.
JavaScript fetch()	Sends inputs to the backend.

2.5 Research Issues/Observations

Machine-learning output depends strongly on data quality and representativeness. The prototype dataset is small and is intended for demonstration. A real healthcare system would require larger validated datasets, privacy controls, independent testing, and professional medical oversight.

2.6 Summary

The discussion establishes why preprocessing, model selection, data quality, and validation are important.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Introduction

System analysis identifies the limitations of a basic manual workflow and defines the proposed software solution.

3.2 Disadvantages/Limitations in the Existing System

Area	Limitation
Manual symptom interpretation	Users may find it difficult to organize several symptoms consistently.
Separated services	Healthcare functions may be presented as unrelated activities.
No automated demonstration	A static interface cannot demonstrate an ML prediction workflow.
Unstructured inputs	Free-form descriptions are harder to process consistently.

3.3 Proposed System

The proposed AI Healthcare Centre integrates patient, doctor, administration, and machine-learning functions. The frontend collects structured symptom inputs. The Flask backend receives JSON, creates a DataFrame with the same feature names used during training, invokes the saved model, and returns the prediction.

3.3.1 Patient Module

Respiratory symptom screening

Appointment booking

Medical-record display

Pharmacy simulation

Shopping cart

Order display

3.3.2 Doctor Module Patient record display

Screening information review

Prescription entry

3.3.3 Administration Module

Doctor management Patient
management

Appointment management

Hospital reports

3.4 Functional Requirements Select a role.

Collect respiratory symptoms and basic demographic information.

Send screening inputs to Flask.

Return a model prediction.

Display the result with a non-diagnostic disclaimer.

Provide appointment, record, pharmacy, cart, and order interfaces.

3.5 Non-Functional Requirements

Usability

Responsiveness

Maintainability

Reliability

Safety and appropriate presentation of screening results

3.6 Summary

The proposed architecture separates presentation, prediction service, and model training.

CHAPTER 4

SYSTEM DESIGN AND IMPLEMENTATION

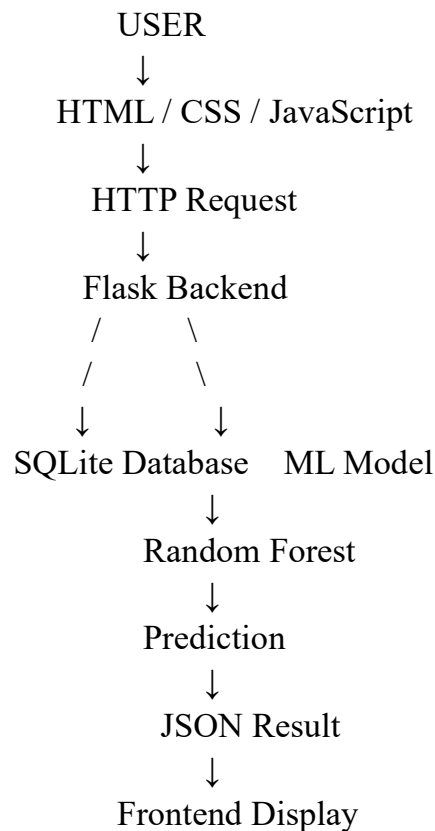
4.1 Introduction

The project is organized around an HTML/CSS/JavaScript frontend, a Flask Python backend, a CSV dataset, and a trained Random Forest model.

4.1 System Architecture

The system follows a layered architecture.

The overall architecture is:



The project documentation describes the architecture as:

User → HTML/CSS/JavaScript Frontend → HTTP/API Requests → Flask Server
→ SQLite Database and ML Model → Random Forest Disease
Screening → Preliminary Result

Component	Technology	Purpose
Frontend	HTML, CSS, JavaScript	User interface and symptom collection.
Backend	Python, Flask	Receives requests and serves predictions.
Data	CSV	Stores labelled training examples.
ML	scikit-learn Random Forest	Disease-label classification.
Serialization	pickle	Stores the trained pipeline.

4.3 Dataset and Data Preparation

Column	Type	Description
Disease	Target	Disease label to predict.
Fever	Categorical	Yes/No symptom.
Cough	Categorical	Yes/No symptom.
Fatigue	Categorical	Yes/No symptom.
Difficulty Breathing	Categorical	Yes/No symptom.
Age	Numerical	Patient age.
Gender	Categorical	Gender category.

The prototype dataset represents Influenza, Common Cold, Asthma, Bronchitis, Pneumonia, Tuberculosis, and Allergic Rhinitis. It is demonstration data rather than a clinically representative dataset.

4.4 Model Development

train_model.py loads dataset.csv, separates Disease as the target, preprocesses categorical features using OneHotEncoder, and trains a Random Forest classifier. The preprocessing and model are combined in a Pipeline and serialized to model.pkl.

4.5 Frontend Implementation

index.html contains the role screens, patient screening form, appointment interface, medical record display, pharmacy/cart simulation, doctor interface, and

administration interface. CSS provides layout and responsive styling, while JavaScript controls interaction.

4.6 Backend Implementation app.py loads model.pkl, starts Flask, exposes a root status endpoint and a /predict POST endpoint. The prediction endpoint receives JSON, creates a pandas DataFrame, calls the model, and returns JSON.

4.7 Integration

The frontend uses JavaScript fetch() to send symptom data to the Flask server running locally on port 5000. CORS is enabled for local development.

File	Purpose
index.html	Main frontend.
backend/dataset.csv	Training dataset.
backend/train_model.py	Model training script.
backend/model.pkl	Saved trained pipeline.
backend/app.py	Flask prediction API.

4.8 Summary

The implementation connects a browser interface to a Python machine-learning service

CHAPTER 5

PERFORMANCE ANALYSIS

5.1 Introduction

Performance analysis focuses on functional correctness and successful communication between the frontend, backend, and model. Clinical performance cannot be inferred from the small demonstration dataset.

5.2 Performance Measures

Test	Expected Result	Status
Load Flask root	Backend status appears.	Passed during development
Train model	model.pkl created.	Passed during development
Send valid input	JSON prediction returned.	To verify in final demo
Open screening	Screening form appears.	Passed during development
Book appointment	Confirmation appears.	Passed during development
Add cart item	Quantity and total update.	Passed during development
Place order	Order appears in My Orders.	Passed during development

5.3 Performance Analysis

The main integration result is that the model can be trained from the CSV dataset and saved as model.pkl, after which Flask loads the model and exposes a prediction endpoint. This confirms the intended technical pipeline. No clinical accuracy, sensitivity, specificity, or real-world diagnostic performance is claimed.

5.4 Limitations of Evaluation Small dataset.

Demonstration rather than clinically validated data.

No independent clinical test set.

No fairness evaluation across demographic groups.
No clinical diagnosis or autonomous medical decision-making.

CHAPTER 6

FRONTEND IMPLEMENTATION

6.1 Frontend Technologies The
frontend uses:

HTML

CSS

JavaScript

HTML provides the structure.

CSS provides the design.

JavaScript provides:

Buttons

Forms

Dashboard interaction

API communication

Screening functionality

Cart functionality

Appointment functionality

6.2 Patient Interface

The Patient dashboard includes:

Respiratory Screening The
patient enters:

Age

Gender

Fever

Cough

Fatigue

Difficulty Breathing

The information is sent to Flask.

Appointments

The patient can:

Select speciality

Select date

Select time

Book appointment Medical

Records The patient can
view available medical-

record information.
Medicine Delivery The
patient can:
Find medical shops
View medicines
Add medicines to cart
Change quantity
View total
Place order

6.3 Doctor Interface

The Doctor Dashboard provides:

Patient information

Appointments

Patient records

Prescription functionality

The doctor can review patient information and create prescription details.

6.4 Administrator Interface

The Administrator dashboard provides:

Doctor Management Patient

Management

Appointment Management

Reports

Statistics

The dashboard can display counts related to doctors, patients and appointments.

CHAPTER 7

FLASK BACKEND

7.1 Introduction

The Flask backend acts as the connection between the frontend and the machinelearning model.

The browser does not directly run the Python model. Instead:

Frontend

↓

Flask ↓

model.pkl

↓

Prediction

↓

Flask

↓

Frontend

7.2 Predict API

The most important API is:

POST /predict

The frontend sends information such as:

```
{  
  "Fever": "Yes",  
  "Cough": "Yes",  
  "Fatigue": "Yes",  
  "Difficulty Breathing": "No",  
  "Age": 25,  
  "Gender": "Female"  
}
```

Flask receives this information and converts it into the format expected by the model.

The model then returns the predicted disease.

7.3 Prediction Workflow The complete process is:

Patient enters symptoms



JavaScript collects values

↓ fetch() sends POST request



Flask /predict



Pandas DataFrame



Preprocessing Pipeline



Random Forest



Predicted Disease



JSON response



Frontend displays result

The project documentation describes this same AI prediction workflow.

7.4 Other Backend APIs

The backend also supports healthcare-management operations such as:

Endpoint	Purpose
/	Backend status
/predict	AI disease prediction
/doctors	Get/add doctors
/doctors/<id>	Delete doctor
/patients	Get/add patients
/patients/<id>	Delete patient
/appointments	Get/add appointments
/appointments/<id>	Delete appointment
/medical-records	Medical record operations
Prescription API	Prescription operations

CHAPTER 8

FUTURE ENHANCEMENT AND CONCLUSION

8.1 Introduction

The AI Healthcare Centre provides a foundation for extending a student-level healthcare prototype into a more complete software system.

8.2 Limitation/Constraints of the System Small symptom dataset.

Limited symptom features.

Demonstration login rather than secure authentication.

Simulated pharmacy and payment features.

No production database.

No clinical validation.

8.3 Future Enhancements

Use a larger, appropriately licensed and clinically validated dataset.

Add more symptom features and input validation.

Compare algorithms using cross-validation and an independent test set.

Add precision, recall, F1-score and confusion matrix where appropriate.

Add secure authentication and role-based authorization.

Use a secure database for patient, doctor, appointment and order data.

Add privacy protections and audit logging.

Improve accessibility and multilingual support.

Add clinician review so AI output remains decision support.

8.4 Conclusion

The AI Healthcare Centre demonstrates integration of a web frontend, Python backend, structured dataset, and machine-learning model into an academic healthcare prototype. The system provides a respiratory symptom screening workflow together with simulated healthcare-management functions. The Random Forest model is trained from structured features, saved as a reusable pipeline, and exposed through a Flask API.

The project demonstrates how data preparation, machine learning, backend APIs, and frontend interfaces can be connected into a coherent application. The current system is a learning and demonstration prototype, not a clinical diagnostic tool.

APPENDIX A

DATASET STRUCTURE

The prototype dataset uses one target column and six input features.

Column	Example Values
Disease	Influenza, Common Cold, Asthma, Bronchitis, Pneumonia, Tuberculosis, Allergic Rhinitis
Fever	Yes / No
Cough	Yes / No
Fatigue	Yes / No
Difficulty Breathing	Yes / No
Age	Integer values
Gender	Male / Female

Example record: Influenza,Yes,Yes,Yes,No,25,Female

For the final submission, document the dataset source and licensing status accurately. If it is a manually created demonstration dataset, describe it as such.

APPENDIX B

KEY SOURCE CODE STRUCTURE

`train_model.py` loads `dataset.csv`, preprocesses the categorical variables, trains Random Forest, and saves `model.pkl`. `app.py` loads `model.pkl`, starts Flask, accepts JSON at `/predict`, creates the expected input DataFrame, predicts the disease label, and returns JSON.

`index.html` contains the user interface and JavaScript `fetch()` request used to communicate with the Flask API.

REFERENCES

1. Project train_model.py
2. Project app.py
3. Project dataset.csv
4. Project index.html
5. Python
6. Flask
7. Flask-CORS
8. Pandas
9. Scikit-learn
10. Random Forest
11. OneHotEncoder
12. Joblib
13. SQLite